Python Visualization Library Documentation

ShadowFox Data Science Internship — Beginner Task

Libraries Covered: Matplotlib & Seaborn

In [1]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

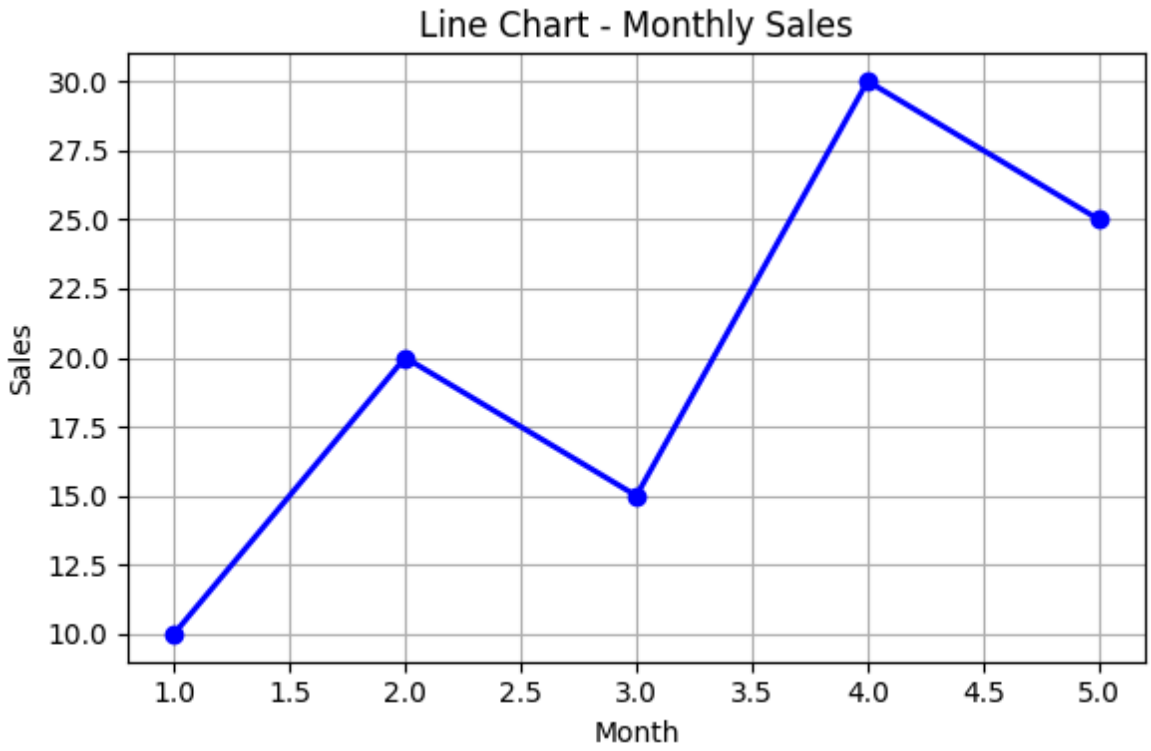
1. MATPLOTLIB

Overview

Matplotlib is a 2D plotting library introduced by John D. Hunter in 2003. Built on NumPy arrays. Foundation for many visualization libraries.

- Use cases: Scientific plots, academic reports, basic data visualization
- Key feature: Full control over every element of a plot

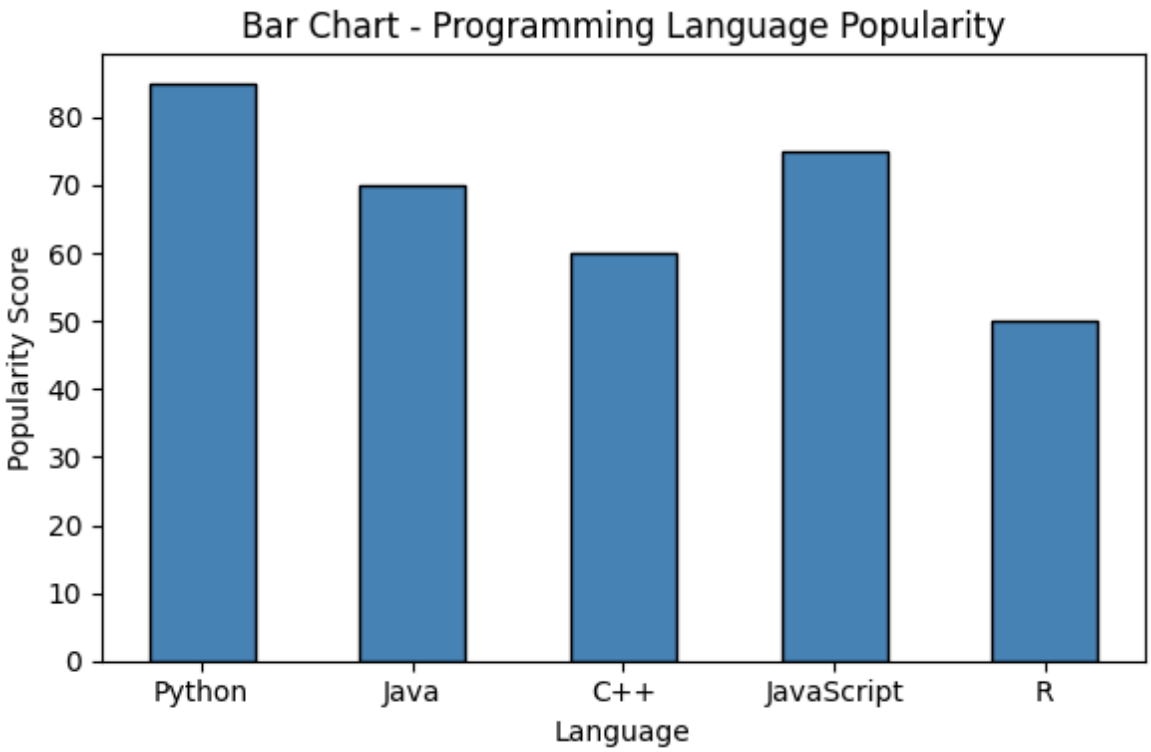
In [2]:
```
x = np.array([1, 2, 3, 4, 5])
y = np.array([10, 20, 15, 30, 25])

plt.figure(figsize=(6, 4))
plt.plot(x, y, color='blue', linewidth=2, marker='o')
plt.title("Line Chart - Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.grid(True)
plt.tight_layout()
plt.show()
print("Use case: Showing trends over time")
```




```
Use case: Showing trends over time
```

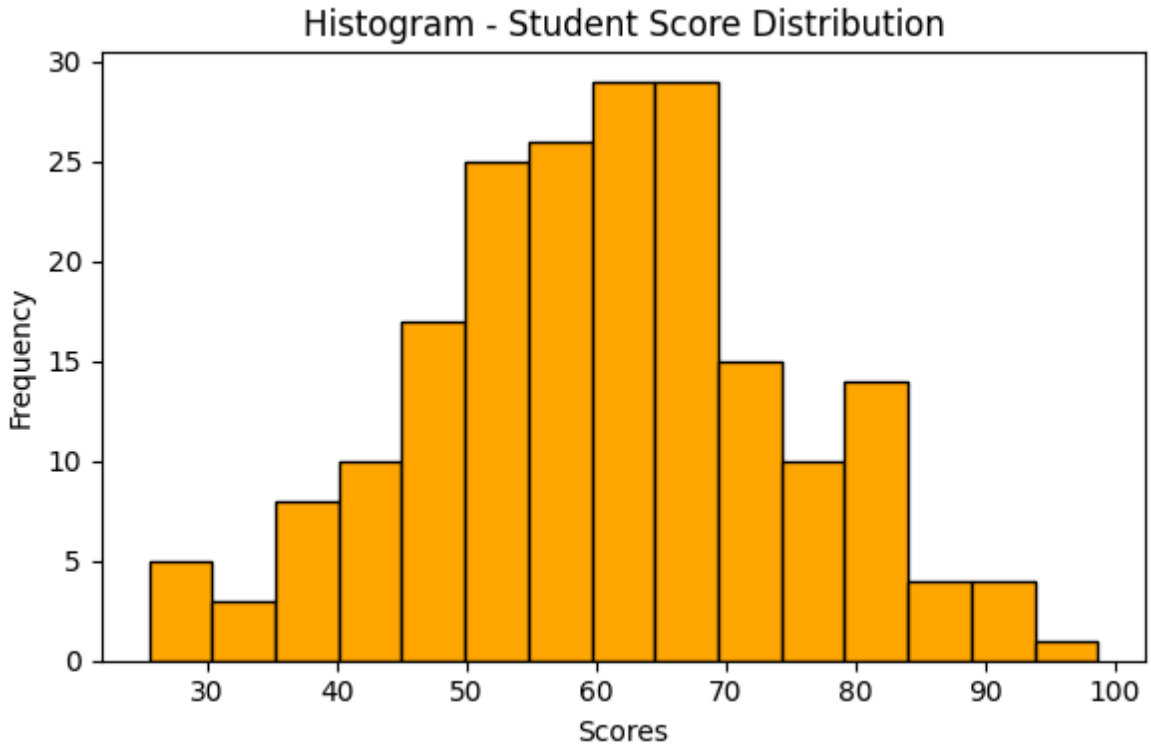
In [3]:

```python
categories = ['Python', 'Java', 'C++', 'JavaScript', 'R']
values = [85, 70, 60, 75, 50]

plt.figure(figsize=(6, 4))
plt.bar(categories, values, color='steelblue', edgecolor='black', width=0.5)
plt.title("Bar Chart - Programming Language Popularity")
plt.xlabel("Language")
plt.ylabel("Popularity Score")
plt.tight_layout()
plt.show()
print("Use case: Comparing categories")
```



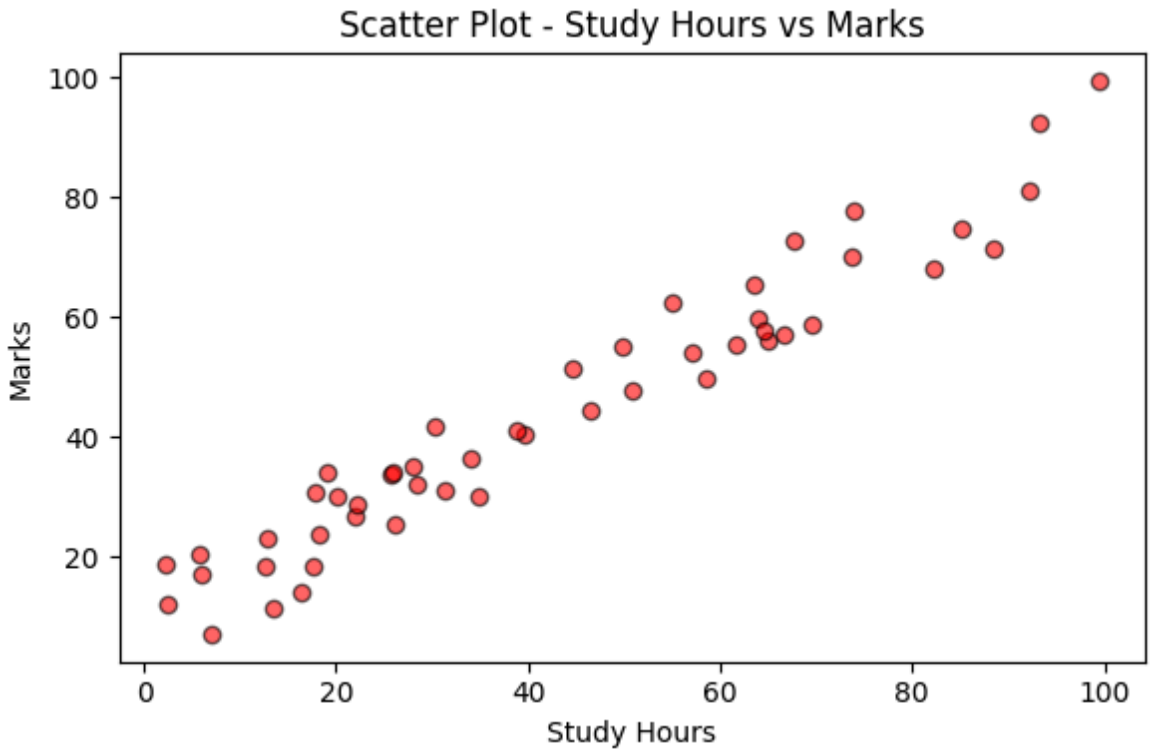
Use case: Comparing categories

In [4]:
```
data = np.random.normal(loc=60, scale=15, size=200)

plt.figure(figsize=(6, 4))
plt.hist(data, bins=15, color='orange', edgecolor='black')
plt.title("Histogram - Student Score Distribution")
plt.xlabel("Scores")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
print("Use case: Frequency distribution of data")
```



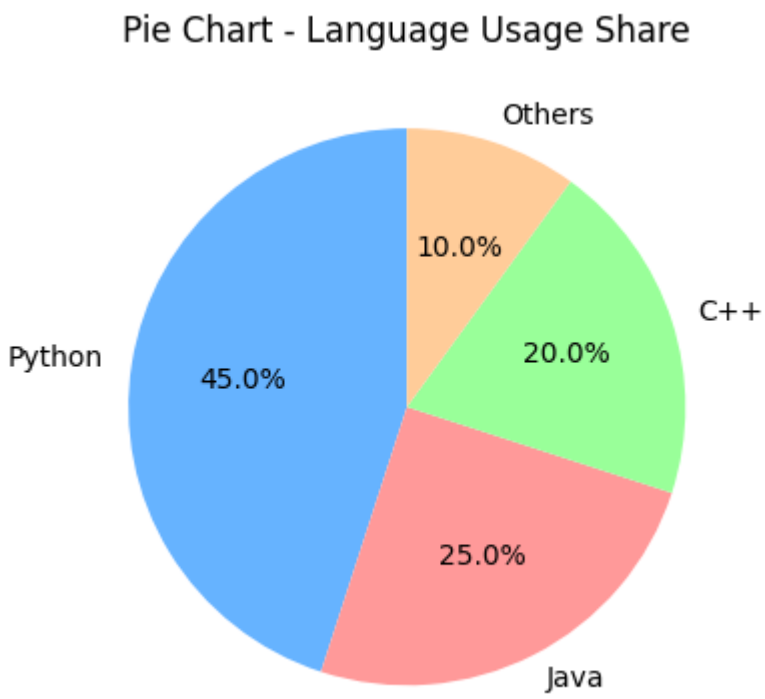
Use case: Frequency distribution of data

In [5]:
```
x = np.random.rand(50) * 100
y = x * 0.8 + np.random.rand(50) * 20

plt.figure(figsize=(6, 4))
plt.scatter(x, y, color='red', alpha=0.6, edgecolors='black')
plt.title("Scatter Plot - Study Hours vs Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
print("Use case: Finding correlation between two variables")
```




```
Use case: Finding correlation between two variables
```

In [6]:
```python
labels = ['Python', 'Java', 'C++', 'Others']
sizes = [45, 25, 20, 10]
colors = ['#66b3ff', '#ff9999', '#99ff99', '#ffcc99']

plt.figure(figsize=(6, 4))
plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=90)
plt.title("Pie Chart - Language Usage Share")
plt.tight_layout()
plt.show()
print("Use case: Showing percentage/proportion of categories")
```




```
Use case: Showing percentage/proportion of categories
```

2. SEABORN

Overview

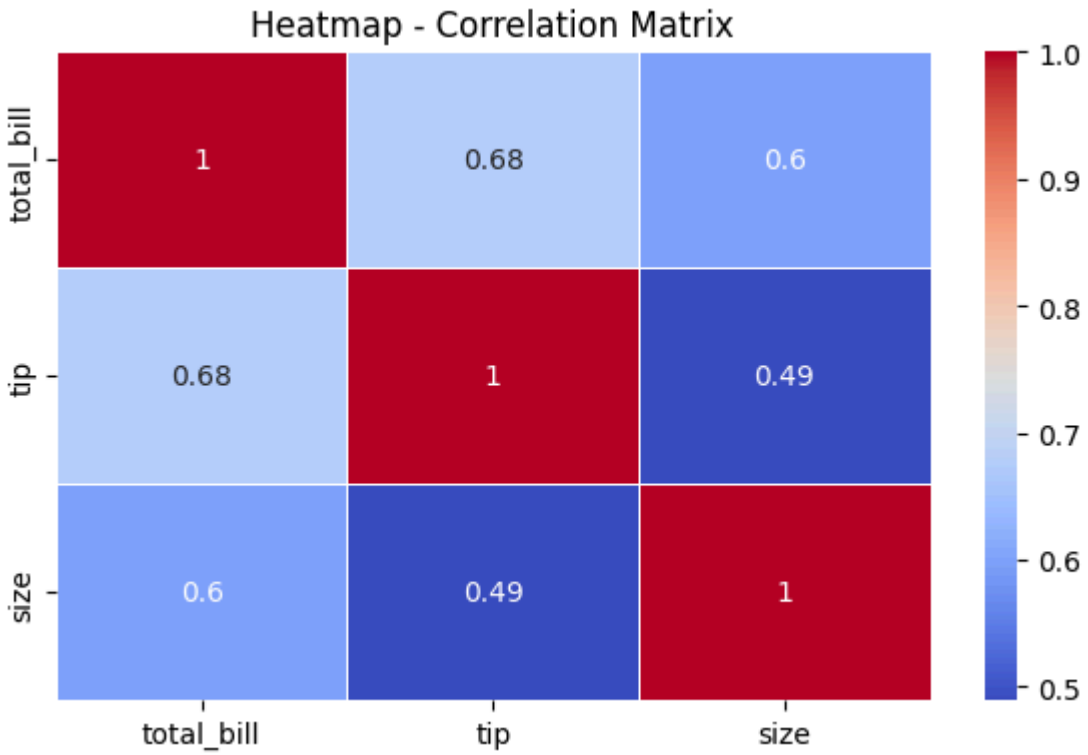
Seaborn is a high-level statistical visualization library built on Matplotlib. Created by Michael Waskom. Produces beautiful plots with minimal code.

- Use cases: Statistical analysis, EDA, correlation analysis
- Key feature: Works directly with Pandas DataFrames

In [7]:
```
tips = sns.load_dataset("tips")
iris = sns.load_dataset("iris")
print("Datasets loaded!")
print(tips.head())
```

```
Datasets loaded!
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
```

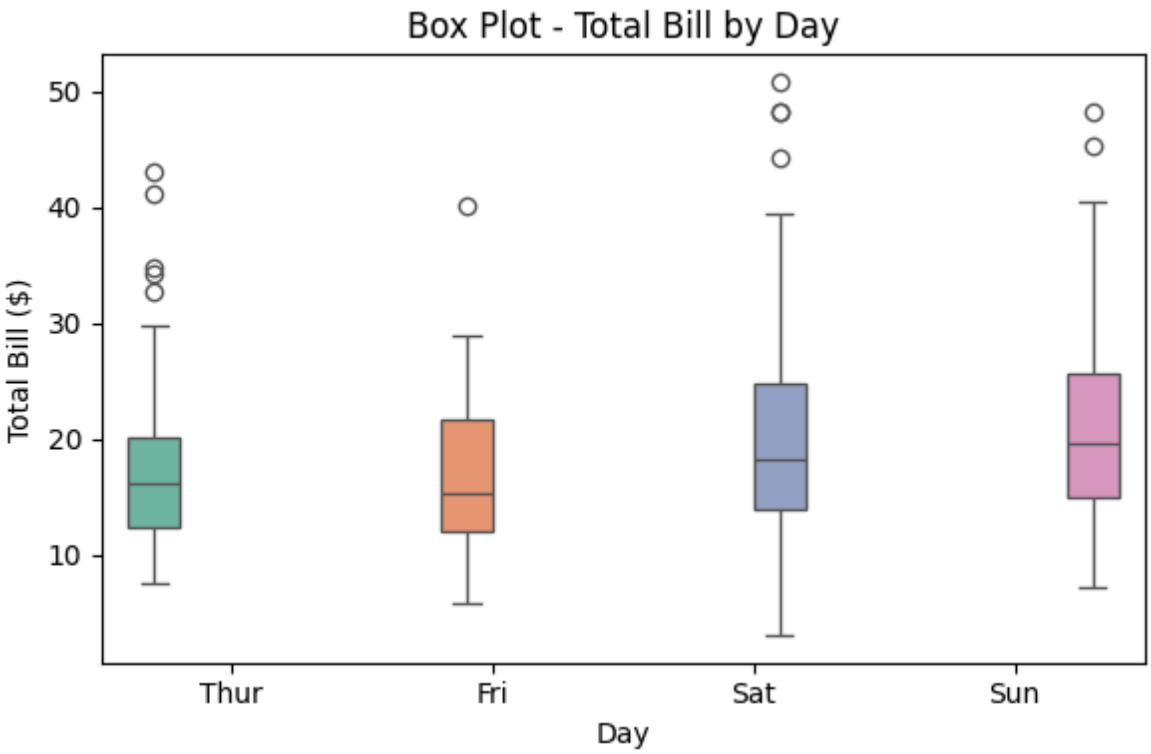
In [8]:
```
plt.figure(figsize=(6, 4))
corr = tips.select_dtypes(include='number').corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title("Heatmap - Correlation Matrix")
plt.tight_layout()
plt.show()
print("Use case: Showing correlation between numerical variables")
```




```
Use case: Showing correlation between numerical variables
```

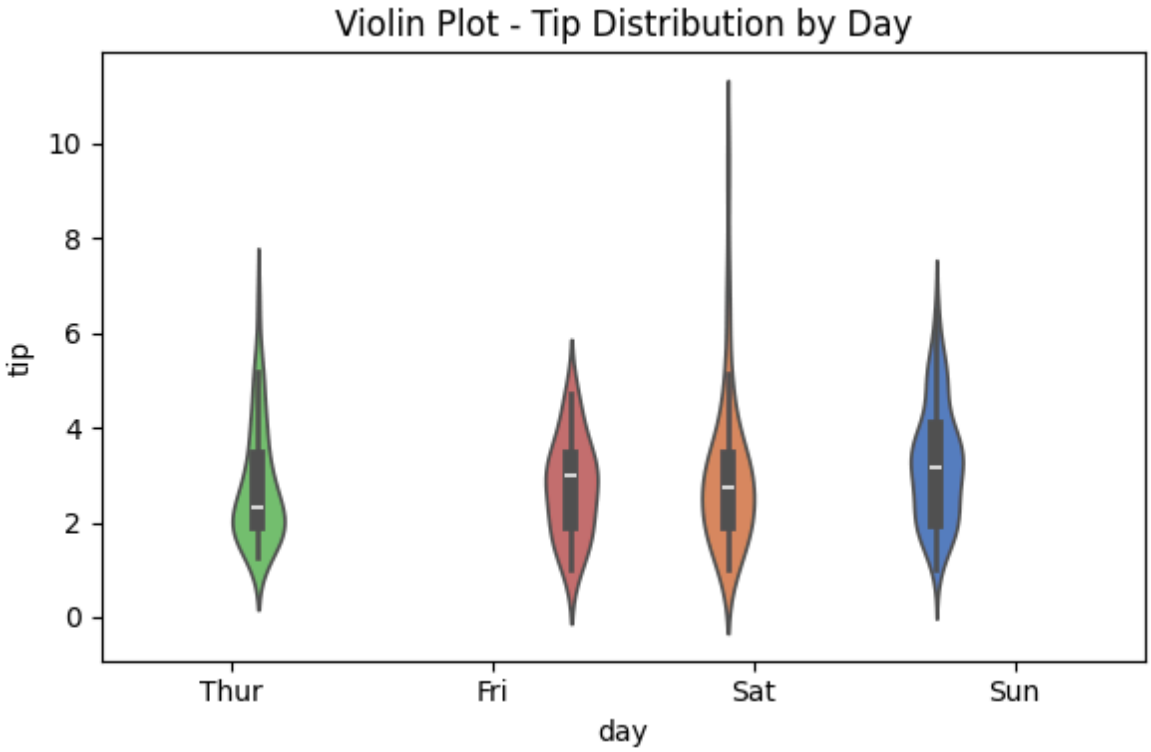
In [9]:

```python
plt.figure(figsize=(6, 4))
sns.boxplot(x='day', y='total_bill', data=tips,
            hue='day', palette='Set2', legend=False)
plt.title("Box Plot - Total Bill by Day")
plt.xlabel("Day")
plt.ylabel("Total Bill ($)")
plt.tight_layout()
plt.show()
print("Use case: Showing data distribution and outliers")
```



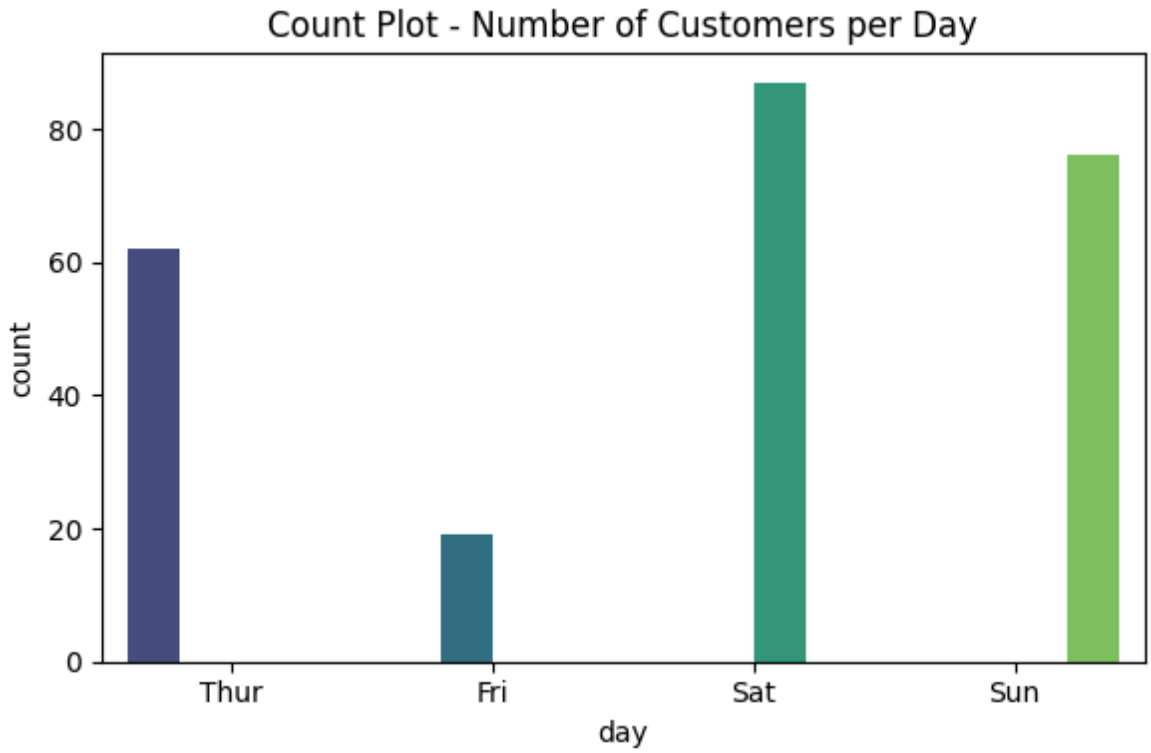

```
Use case: Showing data distribution and outliers
```

In [10]:
```
plt.figure(figsize=(6, 4))
sns.violinplot(x='day', y='tip', data=tips,
               hue='day', palette='muted', legend=False)
plt.title("Violin Plot - Tip Distribution by Day")
plt.tight_layout()
plt.show()
print("Use case: Full distribution shape of data")
```



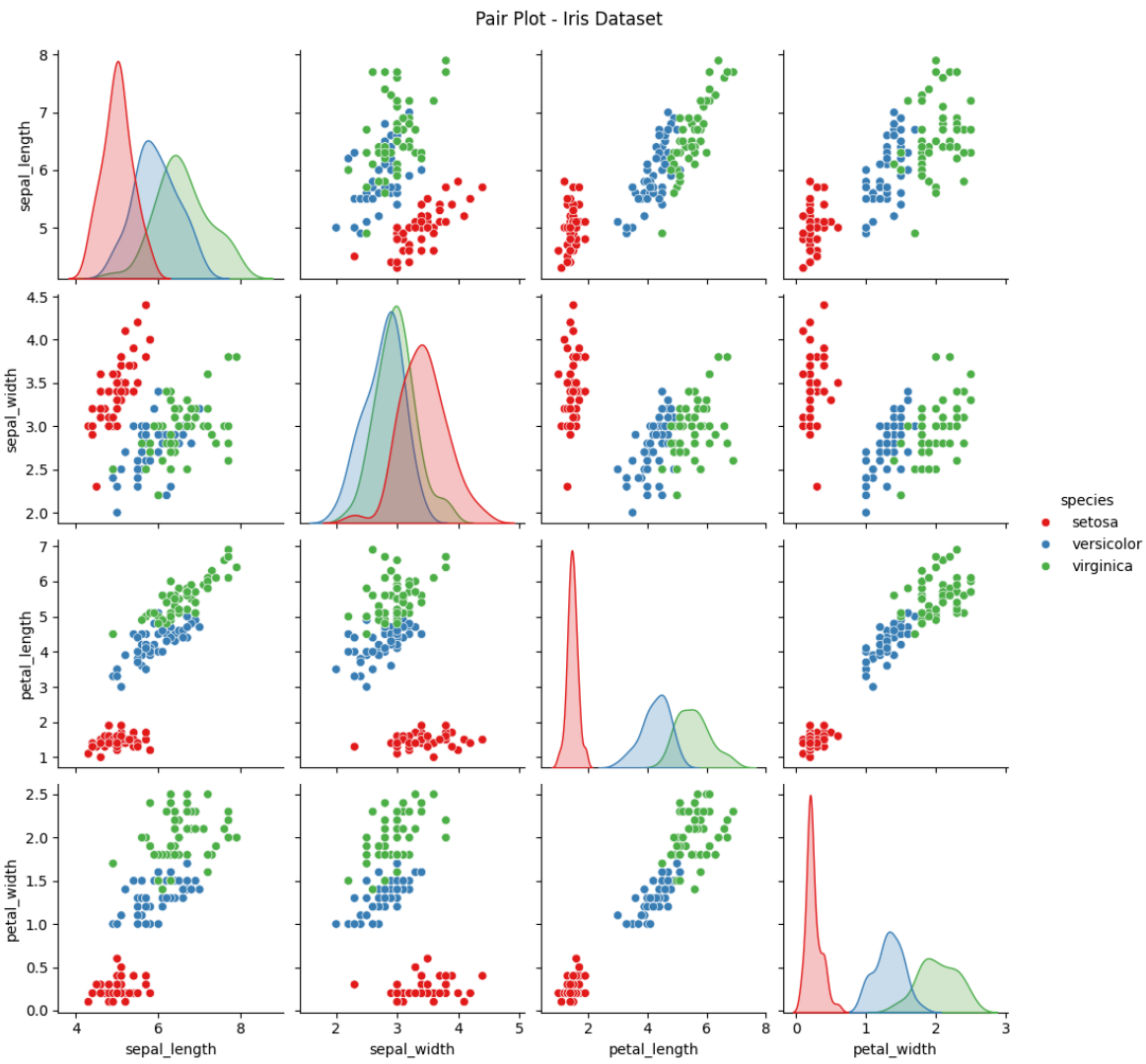

```
Use case: Full distribution shape of data
```

In [11]:
```
plt.figure(figsize=(6, 4))
sns.countplot(x='day', data=tips, hue='day', palette='viridis', legend=False)
plt.title("Count Plot - Number of Customers per Day")
plt.tight_layout()
plt.show()
print("Use case: Count frequency of categorical values")
```




```
Use case: Count frequency of categorical values
```

In [12]:
```
sns.pairplot(iris, hue='species', palette='Set1')
plt.suptitle("Pair Plot - Iris Dataset", y=1.02)
plt.show()
print("Use case: Relationships between all numerical columns")
```




```
Use case: Relationships between all numerical columns
```

3. COMPARISON: Matplotlib vs Seaborn

Feature	Matplotlib	Seaborn
Ease of Use	Medium	Easy
Customization	Very High	Medium
Interactivity	Static	Static
Statistical Plots	Manual	Built-in
Best For	Full control plots	Quick statistical EDA

Conclusion

- Use Matplotlib when you need full control
- Use Seaborn for fast statistical visualization
- Both work best when used together

Sentiment Analysis of Twitter/X Data

ShadowFox Data Science Internship — Intermediate Task

Tool Used: VADER Sentiment Analyzer

In [13]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from wordcloud import WordCloud
import warnings
warnings.filterwarnings('ignore')
print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

In [14]:
```
df = pd.read_csv('twitter_data.csv')
print("Shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nFirst 5 rows:")
df.head()
```

```
Shape: (162980, 2)

Columns: ['clean_text', 'category']

First 5 rows:
```

Out[14]:

	clean_text	category
0	when modi promised “minimum government maximum...	-1.0
1	talk all the nonsense and continue all the dra...	0.0
2	what did just say vote for modi welcome bjp t...	1.0
3	asking his supporters prefix chowkidar their n...	1.0
4	answer who among these the most powerful world...	1.0

1. Data Exploration

In [15]:
```
# Check missing values and category distribution
print("Missing values:\n", df.isnull().sum())
print("\nCategory counts:\n", df['category'].value_counts())
print("\nCategory mapping: -1.0=Negative, 0.0=Neutral, 1.0=Positive")
```

```
Missing values:
 clean_text    4
category      7
dtype: int64

Category counts:
 category
 1.0    72250
 0.0    55213
-1.0    35510
Name: count, dtype: int64

Category mapping: -1.0=Negative, 0.0=Neutral, 1.0=Positive
```

2. Data Cleaning

In [16]:
```
# Drop missing values
df = df.dropna(subset=['clean_text'])

# Map category numbers to labels
df['sentiment'] = df['category'].map({-1.0: 'Negative', 0.0: 'Neutral', 1.0: 'Po

print("Data cleaned!")
print(df['sentiment'].value_counts())
```

```
Data cleaned!
sentiment
Positive    72249
Neutral     55211
Negative    35509
Name: count, dtype: int64
```

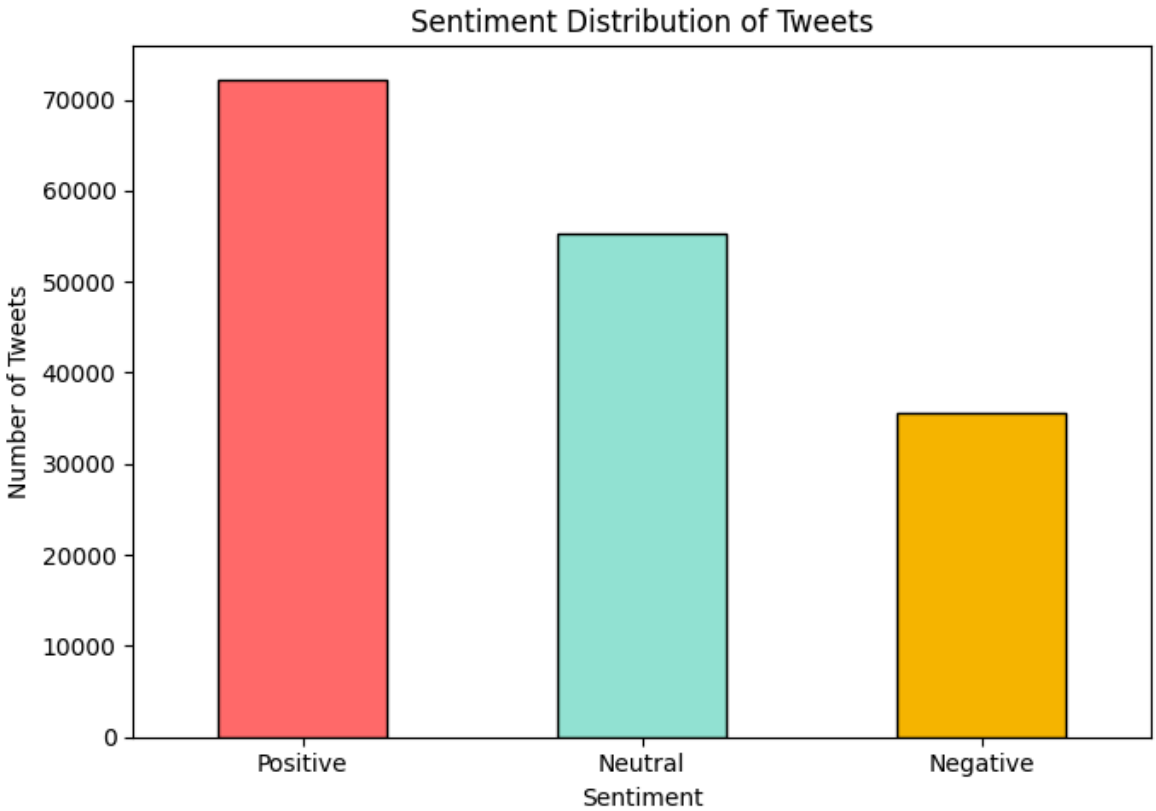
3. VADER Sentiment Analysis

In [17]:
```
# Initialize VADER
analyzer = SentimentIntensityAnalyzer()

# Apply VADER to each tweet
def get_vader_sentiment(text):
    score = analyzer.polarity_scores(str(text))
    if score['compound'] >= 0.05:
        return 'Positive'
    elif score['compound'] <= -0.05:
        return 'Negative'
    else:
        return 'Neutral'

# Apply on sample of 10000 rows (full dataset is too large)
df_sample = df.sample(n=10000, random_state=42)
df_sample['vader_sentiment'] = df_sample['clean_text'].apply(get_vader_sentiment

print("VADER analysis complete!")
print(df_sample['vader_sentiment'].value_counts())
```

```
VADER analysis complete!
vader_sentiment
Positive    4712
Negative    3152
Neutral     2136
Name: count, dtype: int64
```

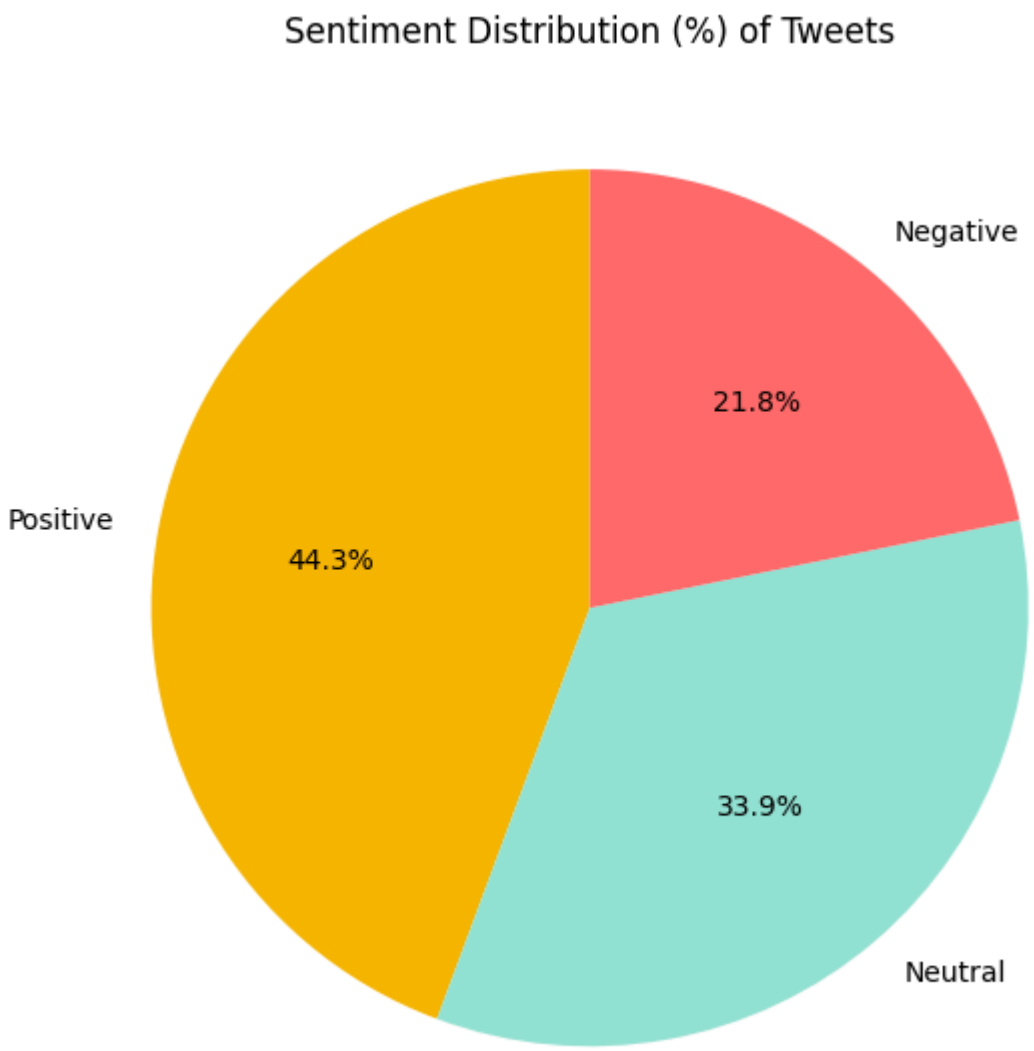
4. Visualizations

In [18]:
```
# Bar chart of sentiment distribution
plt.figure(figsize=(7, 5))
colors = ['#ff6b6b', '#95e1d3', '#f8b500']
df['sentiment'].value_counts().plot(kind='bar', color=colors, edgecolor='black')
plt.title("Sentiment Distribution of Tweets")
plt.xlabel("Sentiment")
plt.ylabel("Number of Tweets")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
print("Total tweets analyzed:", len(df))
```

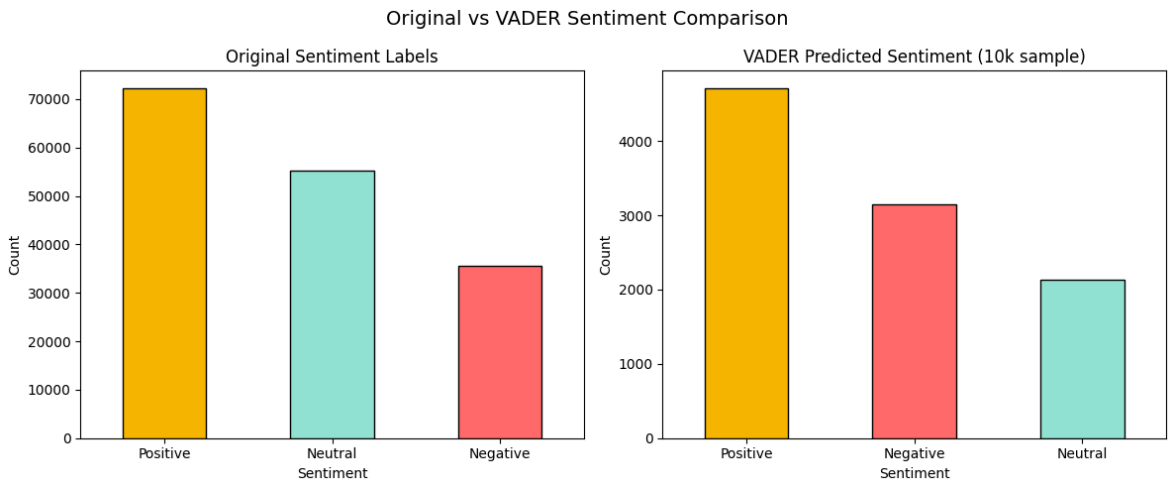



```
Total tweets analyzed: 162976
```

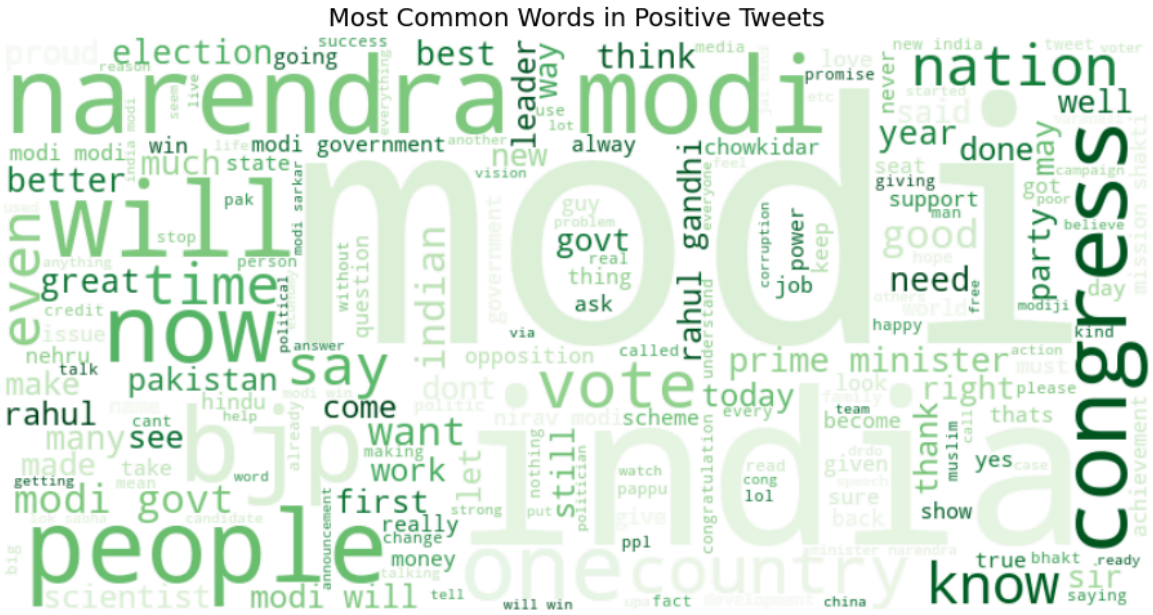
In [19]:
```
# Pie chart
plt.figure(figsize=(6, 6))
sizes = df['sentiment'].value_counts()
colors = ['#f8b500', '#95e1d3', '#ff6b6b']
plt.pie(sizes, labels=sizes.index, colors=colors,
        autopct='%1.1f%%', startangle=90)
plt.title("Sentiment Distribution (%) of Tweets")
plt.tight_layout()
plt.show()
```



In [20]:
```
# Compare original vs VADER sentiment
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Original labels
df['sentiment'].value_counts().plot(kind='bar', ax=axes[0],
    color=['#f8b500', '#95e1d3', '#ff6b6b'], edgecolor='black')
axes[0].set_title("Original Sentiment Labels")
axes[0].set_xlabel("Sentiment")
axes[0].set_ylabel("Count")
axes[0].tick_params(axis='x', rotation=0)

# VADER labels
df_sample['vader_sentiment'].value_counts().plot(kind='bar', ax=axes[1],
    color=['#f8b500', '#ff6b6b', '#95e1d3'], edgecolor='black')
axes[1].set_title("VADER Predicted Sentiment (10k sample)")
axes[1].set_xlabel("Sentiment")
axes[1].set_ylabel("Count")
axes[1].tick_params(axis='x', rotation=0)

plt.suptitle("Original vs VADER Sentiment Comparison", fontsize=14)
plt.tight_layout()
plt.show()
```



In [21]:
```
# Wordcloud for positive tweets
positive_text = ' '.join(df[df['sentiment'] == 'Positive']['clean_text'].dropna(

wordcloud = WordCloud(width=800, height=400,
                      background_color='white',
                      colormap='Greens').generate(positive_text)

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("Most Common Words in Positive Tweets", fontsize=14)
plt.tight_layout()
plt.show()
```

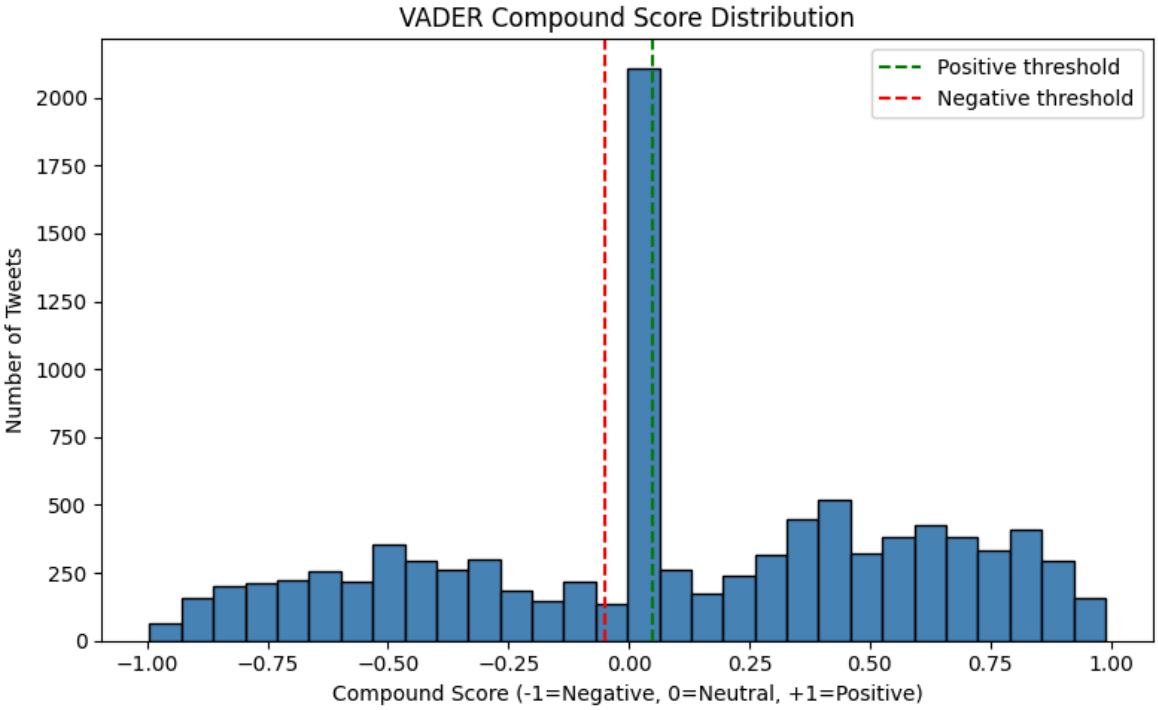

Most Common Words in Positive Tweets
narendra
modi
modi
india
congress
nation
people
will
now
time
great
even
better
election
going
best
think
leader
way
success
win
much
state
modi government
new
alway
chowkidar
gandhi
rahul
year
done
well
may
party
good
need
job
power
corruption
govt
real
thing
ask
indian
say
vote
pakistan
rahul
make
many
see
made
modi
govt
bjp
come
want
work
first
really
change
money
still
one
country
prime
minister
today
right
thank
yes
show
know
sir
saying
true
bhakt
ready
scientist
modi
will
tell
will win
fact
china
opposition
called
scheme
dont
nehru
talk
pak
stop
person
credit
issue
answer
hindu
help
cant
take
word
getting
mean
already
announcement
congratulation
watch
pappu
read
cong
lol
given
sure
back
become
team
look
family
please
thats
muslim
call
achievement
mission
shakti
modiji
kind
action
must
campaign
believe
poor
giving
support
man
seat
said
never
promise
media
tweet
voter
new india
reason
seem
live
modi
sarkar
political
without
question
government
nirav
modi
nothing
politician
put
give
ppl
strong
change
talking
minister
narendra
development
upa
big
candidate
lok sabha
rajya sabha
drdo
speech
case
free
happy
world
hope
attack
understand
everyone
everything
another
use
lot
guy
problem
vision
via
nothing
making
strong
tell
change
money
talking
announcement
already
help
cant
word
mean
take
getting
make
talk
nehru
issue
credit
person
stop
pak
state
much
better
win
modi
modi
india
life
reason
seem
live
modi
sarkar
without
question
political
answer
via
opposition
called
scheme
dont
nirav
modi
nothing
politician
put
give
ppl

```
plt.tight_layout()
plt.show()
```

Most Common Words in Negative Tweets

In [23]:

```
# VADER compound score distribution
df_sample['compound_score'] = df_sample['clean_text'].apply(
    lambda x: analyzer.polarity_scores(str(x))['compound'])

plt.figure(figsize=(8, 5))
plt.hist(df_sample['compound_score'], bins=30,
         color='steelblue', edgecolor='black')
plt.title("VADER Compound Score Distribution")
plt.xlabel("Compound Score (-1=Negative, 0=Neutral, +1=Positive)")
plt.ylabel("Number of Tweets")
plt.axvline(x=0.05, color='green', linestyle='--', label='Positive threshold')
plt.axvline(x=-0.05, color='red', linestyle='--', label='Negative threshold')
plt.legend()
plt.tight_layout()
plt.show()
```



5. Conclusion

- Dataset contains 162,980 tweets with 3 sentiment categories
- Majority of tweets are Positive (44.3%) followed by Neutral (33.9%) and Negative (21.8%)
- VADER Sentiment Analyzer was applied on a sample of 10,000 tweets
- VADER predicted similar trend — more Positive tweets than Negative
- WordCloud shows common words used in Positive and Negative tweets
- VADER compound score distribution shows most tweets lean slightly Positive